

컴퓨터비전 _인공지능 (AI)개론

6. 손실함수 (Loss) 설계

- 손실 함수(Loss/Cost Function)의 기본 개념 및 역할 이해 (오차 측정, 학습 방향 결정)
- 회귀 문제 손실 함수 비교: MSE, MAE 특성 및 이상치 민감도 이해
- Huber Loss 원리 이해: MSE와 MAE 장점을 결합
- 이진 분류: Binary Cross Entropy(BCE) 원리 이해
- Sigmoid와 BCE의 수치적 불안정성 해결: BCEWithLogitsLoss 원리 이해.
- 다중 분류: Cross Entropy Loss 동작 방식
- 모델 과신 완화: Label Smoothing 기법 이해.
- 클래스 불균형 해결 전략 비교: Weighted Cross Entropy와 Focal Loss 이해
- Loss Surface 개념 이해 및 안정적 학습을 위한 좋은 Loss Surface 생성 방법 설명.

기본 개념

손실함수는 모델의 예측값과 실제 정답 사이의 차이를 수치화하는 도구입니다. 이 값이 작을수록 모델이 더 정확하게 예측하고 있다는 의미입니다.

주요목적

- 모델의 오류 정도 측정
- 학습 방향 결정
- 최적화 알고리즘의 가이드

시험 채점 비유

정답과 답안의 차이가 클수록 점수가 낮아지는 것처럼, 손실함수는 예측과 정답의 차이가 클수록 큰 값을 반환합니다.

학습의 나침반

손실함수 값이 감소하는 방향으로 가중치를 조정하면서 모델이 점점 더 정확한 예측을 하도록 학습합니다.

MSE는 가장 널리 사용되는 회귀 손실함수로, 오차를 제곱하여 큰 오차에 더 큰 패널티를 부여

$$\text{MSE} = (1/n) \times \sum (y_{\text{true}} - y_{\text{pred}})^2$$

구체적인 예시

실제값: [10, 20, 30]

예측값: [12, 19, 35]

오차: [2, 1, 5]

제곱: [4, 1, 25]

$$\text{MSE} = (4 + 1 + 25) / 3 = 10$$

장점

- 수학적으로 다루기 쉬움
- 미분 가능
- 빠른 학습 속도

단점

- 이상치에 매우 민감
- 큰 오차 하나가 전체 손실 증가

평균 절대 오차

오차의 절댓값만 계산하여 이상치에 덜 민감한 손실함수입니다.

$$\text{MAE} = (1/n) \times \sum |y_{\text{true}} - y_{\text{pred}}|$$

구체적인 예시

실제값: [10, 20, 30],
예측값: [12, 19, 35]일 때
오차: [2, 1, 5]
절댓값: [2, 1, 5]

$$\text{MAE} = (2 + 1 + 5) / 3 = 2.67$$

- 강건성
- 모든 오차를 동등하게 취급하여 이상치의 영향을 받지 않습니다.
- "평균적으로 2.67만큼 틀림"처럼 직관적으로 해석 가능합니다.
- 0근처에서 미분 불가능(최적화 시 문제)

사용 권장

- 이상치가 많은 데이터
- 해석 가능성 중요
- 공평한 평가 필요

MSE와 MAE의 절충안

- **MSE와 MAE 장점 결합**
- Huber Loss는 작은 오차에는 MSE처럼, 큰 오차에는 MAE처럼 동작하는 영리한 손실함수
- δ (델타: 경계값 하이퍼파라미터) 값을 기준으로 두 방식을 전환합니다.



작은 오차

$|\text{오차}| \leq \delta$ 일 때 제곱 함수 사용. 빠르게 수렴하도록 MSE 방식 적용



큰 오차

$|\text{오차}| > \delta$ 일 때 선형 함수 사용. 이상치에 강건하도록 MAE 방식 적용



균형

빠른 수렴과 강건성을 동시에 확보. $\delta=1.0$ 이 표준 설정값

$\delta = 0.5$

MSE에 가까움
(더 민감)

$\delta = 1.0$

표준 설정
(권장)

$\delta = 2.0$

MAE에 가까움
(더 강건)

손실함수	이상치 민감도	수렴 속도	해석성	추천 상황
MSE	매우 높음	빠름	중간	일반 회귀, 이상치 없음
MAE	낮음	느림	높음	이상치 많음, 해석 중요
Huber	중간	중간	중간	균형 필요, 로봇공학
MAPE	높음	중간	높음	비율 오차 중요
LogCosh	중간	빠름	낮음	MSE 대안

- 두 개의 클래스를 구분하는 손실함수
- BCE는 0 또는 1, 두 가지 클래스를 분류할 때 사용합니다.
- 모델이 예측한 확률(0~1)과 실제 레이블 사이의 차이를 측정합니다.

01

수식 이해하기

$$\text{BCE} = -[y \times \log(p) + (1-y) \times \log(1-p)]$$

y: 실제 레이블 (0 또는 1)

p: 예측 확률 (0~1 사이)

02

정답이 1일 때

예측 확률 0.9 → 손실 0.105 (작음)

예측 확률 0.5 → 손실 0.693 (중간)

예측 확률 0.1 → 손실 2.303 (큼)

03

확신도가 핵심

정답을 확신할수록 낮은 손실, 틀린 답에 확신하면 높은 손실이 부여됩니다.

❏ 수치 안정성 주의: $p=0$ 또는 $p=1$ 일 때 $\log$ 계산이 불안정해집니다. 실무에서는 작은 $\epsilon(1e-7)$ 을 더해 안정성을 확보합니다.

- 경우 1: 실제 레이블이 1일 때

$$\text{BCE} = -\log(p)$$

$$p = 0.9 \rightarrow \text{BCE} = -\log(0.9) = 0.105 \text{ (작은 손실)}$$

$$p = 0.5 \rightarrow \text{BCE} = -\log(0.5) = 0.693 \text{ (중간 손실)}$$

$$p = 0.1 \rightarrow \text{BCE} = -\log(0.1) = 2.303 \text{ (큰 손실)}$$

- 경우 2: 실제 레이블이 0일 때

$$\text{BCE} = -\log(1-p)$$

$$p = 0.1 \rightarrow \text{BCE} = -\log(0.9) = 0.105 \text{ (작은 손실)}$$

$$p = 0.5 \rightarrow \text{BCE} = -\log(0.5) = 0.693 \text{ (중간 손실)}$$

$$p = 0.9 \rightarrow \text{BCE} = -\log(0.1) = 2.303 \text{ (큰 손실)}$$



문제 상황

Sigmoid 활성화 후 BCE를 적용하면 극단적인 값(0 또는 1)에서 수치 불안정이 발생합니다.



해결책

BCEWithLogitsLoss는 Sigmoid와 BCE를 하나로 합쳐 log-sum-exp 트릭으로 안정성을 보장합니다.



추가 장점

더 빠르고 효율적인 계산. 출력층에 Sigmoid를 넣지 말고 raw logits를 사용하세요!

✗ 불안정한 방식

```
output = model(x)
prob = torch.sigmoid(output)
loss = BCE(prob, target)
# 수치 불안정 위험!
```

✓ 안정적인 방식

```
output = model(x)
loss = BCEWithLogitsLoss(
    output, target)
# 내부에서 안전하게 처리
```

핵심: 모델의 출력층에 Sigmoid를 사용하지 마세요. BCEWithLogitsLoss가 내부적으로 모든 처리를 해줍니다.

3개 이상의 클래스 분류에 사용

CrossEntropyLoss는 여러 클래스 중 하나를 선택하는 분류 문제에 사용됩니다.

PyTorch에서는 Softmax가 내장되어 있어 편리합니다.

1



입력 형식 주의

예측: [배치, 클래스 수] 형태의 raw logits

정답: [배치] 형태의 클래스 인덱스

원-핫 인코딩은 필요 없습니다! 클래스 인덱스만 전달하세요.

올바른 사용법

```
predictions = tensor( [[2.0, 1.0, 0.1]])
```

```
target = tensor([0])
```

```
loss = CrossEntropyLoss()( predictions, target)
```

모델 과신 문제

일반적인 학습에서 모델은 정답에 0.99 이상의 확률을 부여하며 지나치게 확신합니다. 이는 일반화 능력을 저하시키고 새로운 데이터에 취약하게 만듭니다.



시험 공부 비유

"이 문제는 100% 답이 1번"이라고 외우면 문제가 조금만 바뀌어도 틀립니다. 융통성이 없는 학습이죠.



라벨 스무딩

정답 레이블을 부드럽게 만들어 모델이 적당한 확신을 갖도록 합니다. 원래 $[1, 0, 0] \rightarrow [0.933, 0.033, 0.033]$

수식과 예시

$$y_{\text{smooth}} = y \times (1 - \alpha) + \alpha / K$$

- α : 스무딩 파라미터 (보통 0.1)

- K : 클래스 개수

클래스 0: $1 \times 0.9 + 0.1/3 = 0.933$

클래스 1,2: $0 \times 0.9 + 0.1/3 = 0.033$

α 권장값: 0.1 (ImageNet 표준)

효과

- 일반화 성능 향상
- 과적합 방지
- 예측 확률의 신뢰도 증가

심각한 데이터 불균형

의료 진단처럼 정상 9,900개(99%), 질병 100개(1%)인 상황을 상상해보세요.
모델이 "모두 정상"이라고만 예측해도 정확도 99%를 달성합니다.
하지만 이는 쓸모없는 모델입니다.

...

가중 교차 엔트로피

소수 클래스의 손실에 더 큰 가중치를 부여합니다. $w_c = \text{전체 샘플 수} / (\text{클래스 수} \times \text{클래스 } c \text{ 샘플 수})$

예시: 클래스 0 (9,900개) → 가중치 0.505
클래스 1 (100개) → 가중치 50.0

Focal Loss - 어려운 샘플에 집중

쉬운 샘플의 손실을 줄이고 어려운 샘플에 집중합니다. $\text{Focal Loss} = -\alpha \times (1-p)^\gamma \times \log(p)$

정답 확률이 0.9면 손실이 거의 무시되고, 0.1이면 손실이 크게 유지됩니다.

+

심각한 데이터 불균형

Focal Loss는 극심한 불균형(1:100 이상)에 효과적이며, 객체 탐지나 의미론적 분할에서 자주 사용됩니다. $\gamma=2$, $\alpha=0.25$ 가 표준 설정입니다.

1	2	3
⋮ 정답 확률 0.9 0.5 0.1	일반 CE 작음 중간 큼	Focal Loss 매우 작음 중간 큼

불균형 대응 전략 완벽 가이드

방법	불균형 정도	난이도	효과	추천 상황
Weighted CE	1:10 ~ 1:100	쉬움	중간	일반적 불균형
Over-sampling	1:5 ~ 1:50	중간	중간	데이터 증강 가능
Under-sampling	1:10 ~ 1:100	쉬움	낮음	데이터 많을 때
Focal Loss	1:100 ~ 1:1000	어려움	높음	극심한 불균형
SMOTE	1:10 ~ 1:50	중간	중간	작은 데이터셋

불균형 정도 파악

먼저 클래스 비율을 계산하여 어느 정도의 불균형인지 확인하세요.

적절한 방법 선택

데이터 크기, 불균형 정도, 구현 난이도를 고려하여 최적의 전략을 선택합니다.

실험과 검증

여러 방법을 시도하고 검증 데이터로 성능을 비교하여 최선의 접근법을 찾으세요.

가중치 공간의 지형도

손실 곡면은 가중치 공간에서 손실값의 변화를 나타낸 지형입니다.
산악 지형처럼 높낮이가 있고, 우리의 목표는 가장 낮은 계곡을 찾는 것입니다.

1



부드러운 곡면

그래디언트가 일관되어 학습이 안정적입니다. 최적값을 찾기 쉽고 수렴이 빠릅니다.

2



넓은 최소값

일반화 성능이 좋고 안정적입니다. 새로운 데이터에도 강건하게 작동합니다.

3



적은 지역 최소값

지역 최소값이 적을수록 전역 최적값을 찾기 쉽습니다. 학습 실패 가능성이 줄어듭니다.

가중치 공간의 지형도

- BatchNorm 사용: 내부 공변량 이동 감소
- Skip Connection: 깊은 네트워크도 부드럽게
- 적절한 초기화: He/Xavier 초기화 ²
- 작은 배치 크기: 넓은 최소값 발견

학습 안정성 확보

- 그래디언트 클리핑: 폭발 방지
- Warm-up: 초반 안전한 탐색
- 적절한 학습률: 진동 방지
- 손실값 모니터링: 이상 징후 조기 발견